



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
APLICACIONES MÓVILES



Integrantes: Melany Lema y Dilan Real

Grupo: 4

Fecha: 17/04/2026

Taller: Arquitectura y Evolución del Desarrollo Móvil

1. Introducción

El desarrollo móvil no solo implica programar, sino elegir la arquitectura correcta, lo cual impacta:

- Rendimiento
- Costo
- Escalabilidad

Objetivo:

Analizar la evolución del desarrollo móvil y entender qué tecnología usar según el contexto.

2. Evolución del Desarrollo Móvil

2.1 Desarrollo Nativo Puro (iOS/Android)

Definición:

Desarrollar aplicaciones para cada plataforma con herramientas y lenguajes nativos, aprovechando directamente el sistema operativo.

Tecnologías:

- iOS: Swift
- Android: Kotlin / Java

Ventajas:

- Máximo rendimiento (60–120 FPS)
- Acceso total al hardware
- Mejor seguridad

Desventajas:

- Alto costo (2 equipos)
- Mantenimiento duplicado

Estado actual: Muy vigente (banca, AR, apps críticas)

2.2 La Era del WebView / Híbridos (Cordova, PhoneGap, Ionic inicial)

Definición:

Empaquetar la aplicación web completa (HTML, CSS, JavaScript) dentro de un WebView embebido, ejecutado como aplicación móvil.

Ventajas:

- Desarrollo rápido
- Bajo costo

Desventajas:

- Bajo rendimiento
- Mala experiencia de usuario

Estado actual: En declive.

2.3 El Puente Nativo-JavaScript y otros puentes

Definición:

La lógica corre en JavaScript; la interfaz es nativa y se comunica mediante un puente entre JS y código nativo.

Cómo funciona:

Usa un “bridge” para comunicarse con el sistema

Ventajas:

- Una sola base de código
- Desarrollo rápido

Desventajas:

- Posibles problemas de rendimiento (bridge)
- Dependencia de módulos nativos

Estado actual: Vigente y estable.

2.4 El Motor de Renderizado Propio

Definición:

Controla todo el renderizado (layout, pintura y composición) sin usar widgets nativos del sistema operativo.

Tecnología:

- Lenguaje: Dart
- Motor gráfico: Skia / Impeller

Ventajas:

- Animaciones fluidas
- UI consistente en todas las plataformas
- Buen rendimiento

Desventajas:

- Mayor tamaño de app
- Integración nativa más compleja

Estado actual: En crecimiento fuerte.

3. Nuevas Tecnologías (Futuro)

3.1 Foldables

Las pantallas plegables requieren aplicaciones capaces de adaptarse dinámicamente a distintos tamaños de pantalla. Esto implica:

- Arquitecturas reactivas
- Manejo eficiente del estado

El desarrollo nativo ofrece el mejor soporte, seguido por Flutter, mientras que React Native presenta mayores limitaciones.

3.2 AR/XR (Realidad Aumentada)

Las aplicaciones de realidad aumentada requieren:

- Renderizado 3D en tiempo real
- Baja latencia

El desarrollo nativo es la mejor opción, ya que permite acceso directo a herramientas como ARKit y ARCore.

Las soluciones multiplataforma dependen de motores externos, lo que limita su rendimiento.

4. Análisis Crítico de Aplicaciones

4.1 Airbnb (Migración desde React Native)

Airbnb inicialmente adoptó React Native para acelerar el desarrollo y compartir código entre plataformas. Sin embargo, enfrentó problemas como:

- Bajo rendimiento en interfaces complejas
- Dificultades con el bridge
- Problemas de mantenimiento

Finalmente, la empresa decidió migrar nuevamente a desarrollo nativo para mejorar el rendimiento y la estabilidad.

4.2 Spotify (Arquitectura Nativa)

Spotify optó por una arquitectura completamente nativa debido a sus requisitos técnicos. Necesidades clave:

- Streaming de audio en tiempo real
- Baja latencia
- Ejecución en segundo plano

El uso de tecnologías nativas permite un control total del hardware y garantiza una experiencia fluida.

5. Análisis Crítico

Para una aplicación que requiera:

- Mapas 3D interactivos
- Notificaciones en tiempo real
- Soporte para pantallas plegables

La mejor opción es el desarrollo nativo.

Esto se debe a su alto rendimiento, acceso directo al hardware y mejor adaptación a nuevas tecnologías.

Aunque React Native y Flutter ofrecen ventajas en productividad, presentan limitaciones en escenarios de alta exigencia.

¿Por qué elegir cada tecnología?

Tecnología	¿Cuándo usar?
Nativo	Alto rendimiento, AR, apps críticas
React Native	Apps de negocio, rapidez
Flutter	UI compleja, animaciones
Híbrido	MVP o apps simples

6. Aplicaciones Nativas

Un aplicativo nativo es aquel desarrollado específicamente para un sistema operativo utilizando herramientas oficiales.

Clasificación:

- iOS: Swift / Objective-C
- Android: Kotlin / Java

Estas aplicaciones ofrecen el mejor rendimiento y la mayor integración con el dispositivo.

7. Conclusión

La evolución del desarrollo móvil demuestra que no existe una única solución ideal. Cada arquitectura responde a necesidades específicas.

El desarrollo nativo continúa siendo la mejor opción para aplicaciones complejas, mientras que las tecnologías multiplataforma son útiles cuando se busca optimizar tiempo y costos.

La clave está en seleccionar la tecnología adecuada según los requerimientos del proyecto.

“La arquitectura correcta depende del problema, no del lenguaje”.